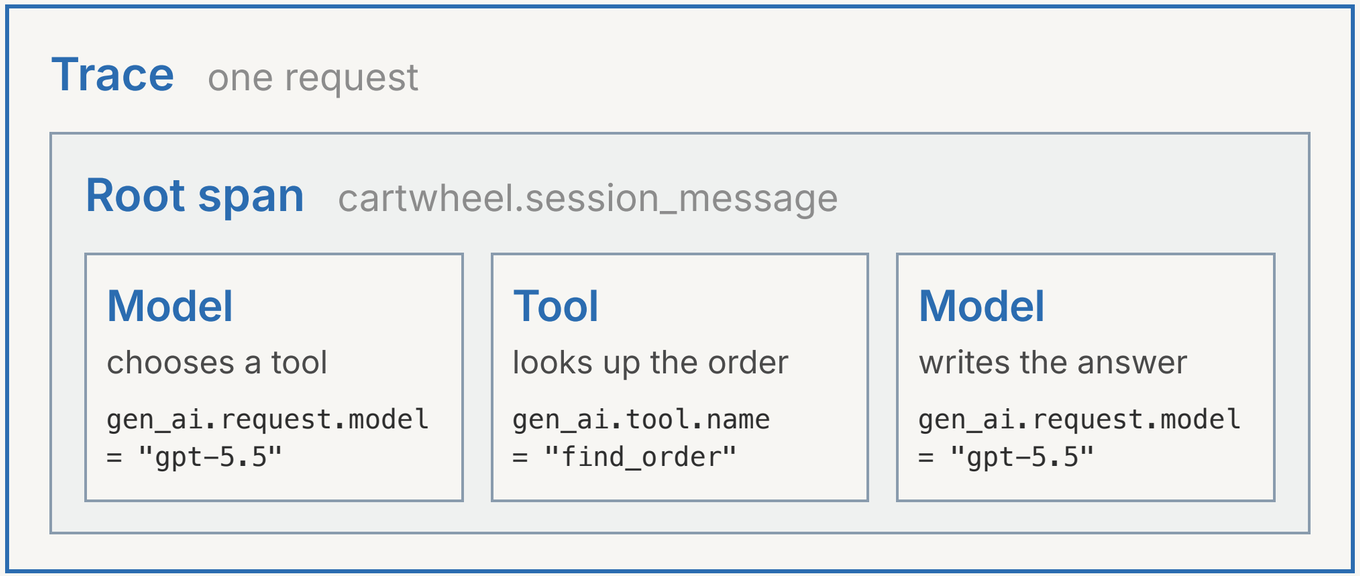
Lesson 3:
Synthetic data and scenarios

Shreya Shankar & Hamel Husain

Where we are

- Instrumented agent, custom viewer
- One confirmed failure from reading a trace
- Next: many more traces, for error analysis and evals



Today

- Generating traces synthetically
- Using a coding agent to help you generate them
- Building a review interface and reading the pilot traces
- Homework 3

Naive way to synthetically generate traces

Ask a coding agent to generate the requests:

PROMPT

Generate 250 support questions for an e-commerce platform.

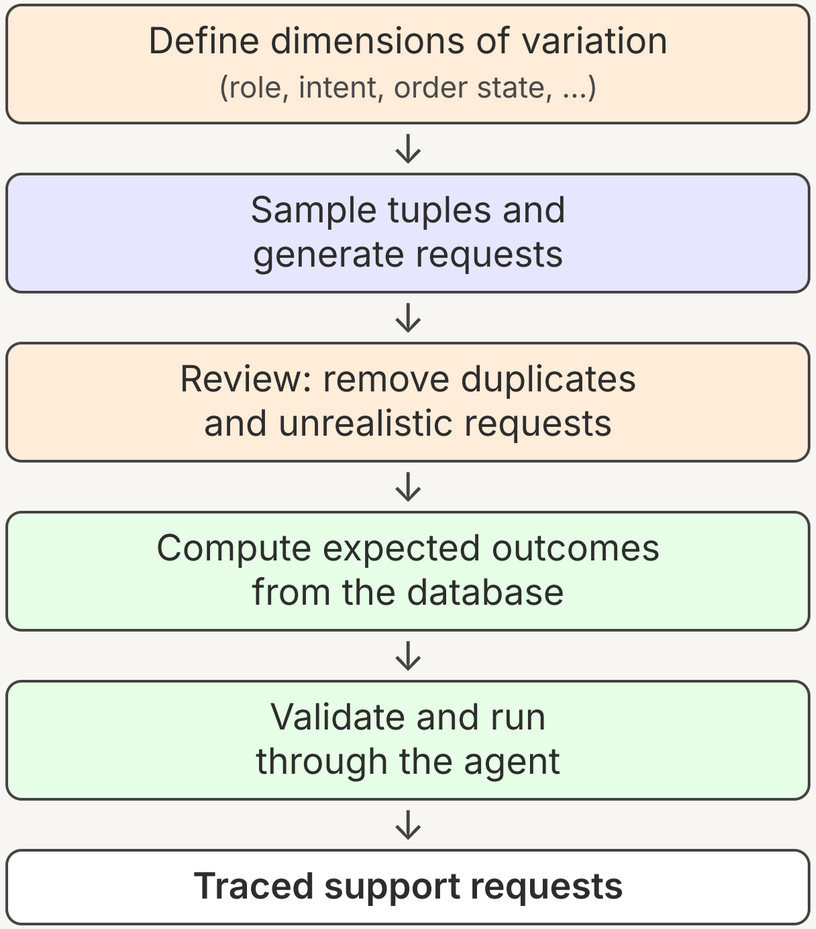
What it gives us:

- "Where is my order?"
- "Can you check the status of my order?"
- "I placed an order last week and haven't received it yet."
- "What's the delivery date for order #4127?"

Little diversity: all four are order status. Nothing about, e.g., refunds, store overrides, or merchants.

Our approach to synthetic data generation

- **Key idea:** use the agent to generate scenarios for user requests, not user requests directly
- **Scenario:** a planned user request plus its expected result
- Human steps in orange, model steps in blue, code steps in green
- Expected results from the database and the rules, not from the model



Step 1: Define dimensions of variation

Dimensions → Tuples, requests → Review → Expected outcomes → Validate, run → Traces

Dimension: a named source of variation, read from the specification

Dimension	Values
Role	shopper, merchant, support
Intent	order status, refund, cancellation, policy question, product search, dispute, out of scope
Record involved	an order (in window, past window, above threshold, placed, shipped), a product, a store policy, none
Applicable policy	platform rule, store override, none
Tools needed	none, one lookup, several calls
Difficulty	well specified, ambiguous, missing information, boundary
User style	neutral, terse, typo-heavy, rambling, frustrated, pressuring, operational shorthand, asks for a short answer

Step 2: Sample tuples and generate requests

Dimensions → **Tuples, requests** → Review → Expected outcomes → Validate, run → Traces

Tuple: one value per dimension

Role	shopper
Intent	refund
Record	an order past the return window
Policy	platform rule
Tools	several
Difficulty	ambiguous
User style	confused, rambling

Request: the message a user with that tuple would send

> "Can I return the vase I got a while ago? I never used it."

Generated from the tuple by a model. No order number, no date, no policy in the message.

Prompt for tuples

Dimensions → **Tuples, requests** → Review → Expected outcomes → Validate, run → Traces

- ✦ Ask a model for combinations, with a few hand written tuples as examples

PROMPT

Generate 10 random combinations of (role, intent, record, policy, tools, difficulty, user style) for a Cartwheel support agent.
Role: shopper, merchant, support. Intent: order status, refund, cancellation, policy question, product search. Record: an order and its state, a product, a store policy page, none. Policy: platform rule, store override, none. Tools: none, one lookup, several. Difficulty: well specified, ambiguous, missing information. User style: neutral conversational, terse or fragmentary, typo-heavy, confused or rambling, frustrated or impatient, repetitive or pressuring, operational shorthand, requests a short plain-language answer.

Examples:
(shopper, refund, order past the window, platform rule, several, ambiguous, confused or rambling)
(merchant, order status, order placed, none, one lookup, well specified, operational shorthand)

Output each tuple on one line. Avoid duplicates. Vary values across all dimensions.

Prompt for the request

Dimensions → **Tuples, requests** → Review → Expected outcomes → Validate, run → Traces

- For every tuple, a model writes the request, grounded in a real database row

PROMPT

You are simulating a customer messaging the Cartwheel support agent. Write a realistic one sentence message from the user described below. Write naturally, as if the user is typing into a chat window. Do not state every fact from the tuple. Do not mention the order number.

Role: shopper
Intent: refund
Record: order 3980, delivered 45 days ago, $52, Blue Heron Ceramics
Policy: platform rule
Tools: several
Difficulty: ambiguous
User style: confused, rambling

What it gives us:

- "Can I return the vase I got a while ago? I never used it."

Coverage set and challenge set

Dimensions → **Tuples, requests** → Review → Expected outcomes → Validate, run → Traces

Coverage set, 175 scenarios

Every dimension value appears, ordinary requests included

- "Where's the thing I ordered last week?" order status
- "Can I see what sold in my shop today?" merchant, own store
- "When do I get paid for last week's sales?" policy question

Challenge set, 75 scenarios

Intentionally difficult: policy boundaries, damaged records, pressure on permissions

- "I want to return the teapot I got from Juniper Home Goods, it's been under a month." store override, 14 days
- "Pull up the details on that ceramics order, the blue vase one." merchant of another store
- "Can I still send back the pencil set I got from Atlas Stationery?" missing delivery date

Step 3: Review the requests

Dimensions → Tuples, requests → **Review** → Expected outcomes → Validate, run → Traces

- Independent critic pass for repetitive or template-like language
- Unrealistic wording, e.g., "Please process a refund for order 3980 under the 30 day return policy"
- Impossible combinations, e.g., support staff asking about their own orders
- A person reads complete conversations before the application runs

Step 4: Compute expected outcomes

Dimensions → Tuples, requests → Review → **Expected outcomes** → Validate, run → Traces

For the request from step 2

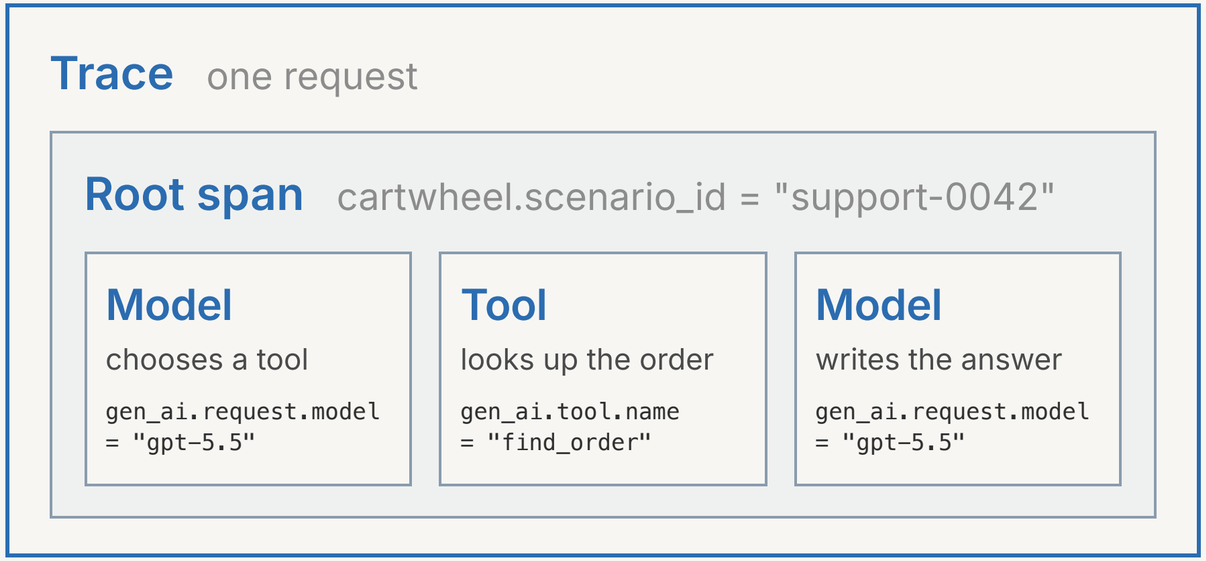
Request	"Can I return the vase I got a while ago? I never used it."
Row	order 3980, delivered 2026-05-17, Blue Heron Ceramics, no override
Rule	30 days from delivery (facts.yaml)
Expected	refund denied, 45 days
Source	the return policy (facts.yaml) and the order row

- **Expected outcome:** what the agent should do, computed from the row and the rule
- Sources: the database, the policy documents, the data quality table
- No single right answer, e.g., was the refusal clear?

Step 5: Validate and run through the agent

Dimensions → Tuples, requests → Review → Expected outcomes → **Validate, run** → Traces

- Validate the file: shape, duplicates, group counts
- Run every request through the agent
- The agent is instrumented, so each run is a Langfuse trace tagged with its scenario id
- The traces are the dataset for Module 2



The synthetic data skill

The workflow above, encoded as a skill for your coding agent

1. Propose dimensions, show the human
2. Coverage pool, challenge pool
3. Ground expected outcomes
4. Generate conversations concurrently
5. Independent critic calls
6. Write and validate conversation plans
7. Human reviews complete conversations
8. Run plans through the application
9. Verify and export traces

scenarios/skill/SKILL.md

Live: the skill

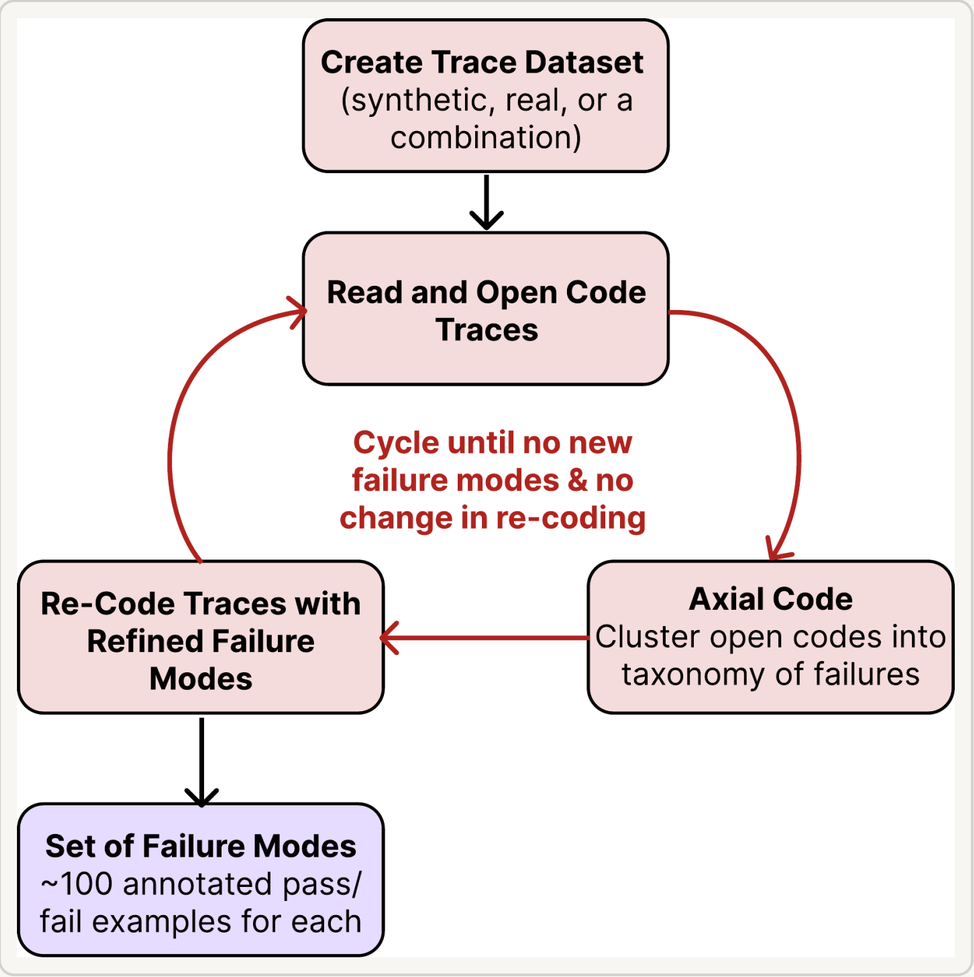
Generate scenarios with a coding agent, then run them

The prompt

> Read `scenarios/skill/SKILL.md` and `SPEC.md`. Query the `data_quality_cases` table. Propose the scenario dimensions and their possible values, then stop so I can review them before generation.

- Check the proposed dimensions
- Read a few of the generated scenarios
- Run the validator

Error analysis



- We just created the trace dataset
- Now we do the second step: read and open code traces
- Roots in qualitative research
- Lecture 4 continues with axial coding and the full taxonomy

Open coding

- Read each trace top to bottom: user messages, tool calls, tool results, final answer
- Stop at the first step where the agent did the wrong thing
- Write a note in your own words: what happened and what should have happened
- No categories yet: describe before you classify

Vibe-coding a custom UI for open coding

Ask a coding agent to build a trace review interface over the pilot traces:

PROMPT

Read `analysis/skill/SKILL.md`. I have pilot traces in Langfuse on port 3001. Build a single-page review interface that shows each trace with its conversation, tool calls and results, and the expected outcome from the scenario file. Add a text field where I can write an annotation for each trace. Serve it on port 8020.

- Conversation messages in order
- Tool calls beside their results
- Expected outcome from the scenario
- A text annotation field, saved to `analysis/state/annotations.json`

Homework 3

Goal: build the scenario and trace dataset for error analysis in Module 2

- **A** Plan the dimensions with the skill
- **B** Pilot: 30 scenarios, at least five confirmed failures
- **C** Final set: 175 and 75, reviewed
- **D** Run 250 on one model: about $6.71, 34 minutes
- **E** Summarize the run and export the traces

Skipped Homework 2? `git apply homework/module-1/hw2-reference.patch`

Homework 3 instructions

Before Lesson 4 (Wednesday 9/16)

- **HW3:** generate scenarios, run the pilot, run the final set, export the traces
- **Read:** Module 1, "Creating an initial scenario dataset" in the course reader
- **Lecture 4:** more open coding and axial coding over 100 traces

When	What
Sat 9/12, 10:30 AM PT	Office hours
Wed 9/16, 8 AM PT	Lecture 4: Error Analysis, Finding Failures
Wed 9/16, 9:30 AM PT	Office hours
Sat 9/19, 11 AM PT	Lecture 5: Error Analysis, Measuring with Evaluators

Please ask questions in Discord!